Documentación Analizador Sintáctico y Semántico

Mario Castro Esteban, Guillermo Delgado Sáez, David Fernández Canedo

Definición del código completo:

```
code: program_definition
      const_section
      var_section
      begin_section
      instruction_section
      end_section
```

- **Code** define la estructura general de un programa Pascal.
- **Program_definition** especifica la declaración del programa.
- **Const_section** y **var_section** son secciones opcionales para constantes y variables.
- **Begin_section** marca el inicio del bloque de código.
- **Instruction_section** contiene las instrucciones del programa.
- **End_section** indica el fin del programa.

Declaración del Programa:

```
program_definition: PROGRAM ID SEMICOLON
      {
        printf("Apertura de programa definida.\n");
      }
      ;
```

- Define la apertura del programa y requiere la palabra clave **PROGRAM**, seguida de un identificador **ID** y un punto y coma **;**.

Sección de Constantes:

```
const_section: CONST
               const_list
      {
        printf("Sección de constantes.\n");
      }
      | /* Empty */
      ;
```

- La sección de constantes **const_section** comienza con la palabra clave **CONST** y puede contener una lista de constantes **const_list**. Puede estar vacía.

Lista de Constantes:

```
const_list: const_list const_item
      | const_item
      ;
```

- Una lista de constantes puede ser una concatenación de múltiples elementos de constante **const_item**.

Elemento de Constante:

```
const_item: ID IGUAL CTE_CADENA SEMICOLON
      {
        printf("Declaración de CONSTANTE DE CADENA.\n");
        addSymbol($1, $3, 1);
        initializeSymbol($1);
      }
      | ID IGUAL CTE_CHAR SEMICOLON
      {
        printf("Declaración de CONSTANTE DE CHAR.\n");
        addSymbol($1, $3, 1);
        initializeSymbol($1);
      }
      | ID IGUAL CTE_INT SEMICOLON
      {
        printf("Declaración de CONSTANTE ENTERA.\n");
        addSymbol($1, $3, 1);
        initializeSymbol($1);
      }
      | ID IGUAL CTE_REAL SEMICOLON
      {
        printf("Declaración de CONSTANTE REAL.\n");
        addSymbol($1, $3, 1);
        initializeSymbol($1);
      }
      | /* Empty */
      ;
```

- Un elemento de constante **const_item** define la asignación de un identificador a un valor constante (cadena, carácter, entero o real).

Sección de Variables:

```
var_section: VAR
             var_list
      {
        printf("Sección de variables.\n");
      }
      | /* Empty */
      ;
```

- La sección de variables **var_section** comienza con la palabra clave **VAR** y puede contener una lista de variables **var_list**. Puede estar vacía.

Lista de Variables:

```
var_list: var_list var_item
        | var_item
        ;
```

- Una lista de variables puede ser una concatenación de múltiples elementos de variable **var_item**.

Variable:

```
var_item: ID DOS_PUNTOS INTEGER SEMICOLON
      {
            printf("Declaración de INTEGER\n");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS REAL SEMICOLON
      {
            printf("Declaración de REAL\n");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS STRING SEMICOLON
      {
            printf("Declaración de STRING\n");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS BOOLEAN SEMICOLON
      {
            printf("Declaración de BOOLEAN\n");
            addSymbol($1, $3, 0);
      }
      | /* Empty */
      ;
```

- Una variable **var_item** define la declaración de una variable con un identificador y su tipo.

Inicio del Bloque de Código:

```
begin_section: BEGIN_KEYWORD
      {
        printf("Inicio de bloque de código.\n");
      }
      ;
```

- Marca el comienzo del bloque de código con la palabra clave **BEGIN**.

Instruction_section:

```
instruction_section: instruction_section instruction
      | instruction
      ;
```

- La sección de instrucciones **instruction_section** puede ser una secuencia de múltiples instrucciones.

Instrucción:

```
instruction: assignment SEMICOLON
      {
        printf("Instrucción de asignación.\n");
      }
      | write_instruction SEMICOLON
      {
        printf("Instrucción de escritura.\n");
      }
      | read_instruction SEMICOLON
      {
        printf("Instrucción de lectura.\n");
      }
      | if_sentence SEMICOLON
      {
        printf("Sentencia condicional.\n");
      }
      | while_sentence SEMICOLON
      {
        printf("Sentencia bucle WHILE.\n");
      }
      | for_sentence SEMICOLON
      {
        printf("Sentencia bucle FOR.\n");
      }
      | /* Empty */
      ;
```

- Define diferentes tipos de instrucciones que pueden aparecer en el programa Pascal, como asignaciones, escrituras, lecturas, sentencias condicionales **if_sentence**, bucles while **while_sentence** y bucles for **for_sentence**.

Assignment:

```
assignment: ID EQUALS expression
      {
        if (findSymbol($1) == -1){
            printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
            conteoErrores++;
        } else {
          if (getSymbolType($1) != $3){
             printf("Línea %d, ERROR SEMÁNTICO: Tipos incompatibles en la asignación de '%s'.\n", conteoLineas+1, $1);
             conteoErrores++;
          }
        }


       if (isInitialized($1) == 0)
          initializeSymbol($1);

       if (isConstant($1) == 1){
          printf("Línea %d, ERROR SEMÁNTICO: Las constantes no se pueden redefinir\n", conteoLineas+1);
          conteoErrores++;
        }
      }
      | ID EQUALS text_expression {
        if (findSymbol($1) == -1){
            printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
            conteoErrores++;
        } else {
          if (getSymbolType($1) != $3){
             printf("Línea %d, ERROR SEMÁNTICO: Tipos incompatibles en la asignación de '%s'.\n", conteoLineas+1, $1);
             conteoErrores++;
          }
        }

       if (isInitialized($1) == 0)
          initializeSymbol($1);

       if (isConstant($1) == 1){
          printf("Línea %d, ERROR SEMÁNTICO: Las constantes no se pueden redefinir\n", conteoLineas+1);
          conteoErrores++;
        }
      }
      ;
```

- Define la asignación de un valor a una variable o constante.

Write_instruction:

```
      ;

write_instruction: WRITELN A_PARENTESIS writeln_arguments C_PARENTESIS
      ;
```

- Define la instrucción de escritura, que utiliza **WRITELN** y puede incluir argumentos a imprimir.

Read_instruction:

```
read_instruction: READLN A_PARENTESIS ID C_PARENTESIS
        {
          if (isInitialized($3) == 0)
            initializeSymbol($3);
        }
      ;
```

- Define la instrucción de lectura **READLN** que lee un valor y lo asigna a una variable.

If_sentence:

```
if_sentence: IF A_PARENTESIS boolean_expression C_PARENTESIS THEN
             BEGIN_KEYWORD
             instruction_section
             END_KEYWORD
           | IF A_PARENTESIS boolean_expression C_PARENTESIS THEN
             BEGIN_KEYWORD
             instruction_section
             END_KEYWORD
             ELSE
             BEGIN_KEYWORD
             instruction_section
             END_KEYWORD
            ;
```

- Define la estructura de una sentencia condicional **IF**. Puede tener o no una rama **ELSE**.

While_sentence:

```
while_sentence: WHILE A_PARENTESIS boolean_expression C_PARENTESIS
                BEGIN_KEYWORD
                instruction_section
                END_KEYWORD
               ;
```

- Define la estructura de un bucle `WHILE`.

For_sentence:

```
for_sentence: FOR A_PARENTESIS ID EQUALS CTE_INT TO CTE_INT C_PARENTESIS
              BEGIN_KEYWORD
              instruction_section
              END_KEYWORD {
                if(getSymbolType($3) != 1){
                   printf("Línea %d, ERROR SEMÁNTICO: Los identificadores usados en las sentencias FOR deben ser siempre de tipo entero\n", conteoLineas+1);
                   conteoErrores++;
                }
              }
            | FOR A_PARENTESIS ID EQUALS CTE_INT DOWNTO CTE_INT C_PARENTESIS
              BEGIN_KEYWORD
              instruction_section
              END_KEYWORD {
                if(getSymbolType($3) != 1){
                   printf("Línea %d, ERROR SEMÁNTICO: Los identificadores usados en las sentencias FOR deben ser siempre de tipo entero\n", conteoLineas+1);
                   conteoErrores++;
                }

              }
             ;
```

- Define la estructura de un bucle **FOR**. Puede tener una condición de incremento **TO** o decremento **DOWNTO**.

Expression_list:

```
expression_list: expression_list COMA expression
      | expression
      ;

expression: term {
        $$ = $1;
      }
      | expression SUMA term
      | expression RESTA term
      ;
```

- Define la lista de expresiones separadas por comas.
- Define expresiones aritméticas, incluyendo operaciones de suma y resta.

Term:

```
term: factor {
        $$ = $1;
      }
      | term MULT factor
      | term DIV factor
      | term MOD factor
      ;
```

- Define términos en expresiones aritméticas, incluyendo multiplicación, división y módulo.

Factor:

```
factor: ID
      {
        if (findSymbol($1) == -1){
          printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
          conteoErrores++;
        } else if (isInitialized($1) == 0){
          printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no inicializado.\n", conteoLineas+1, $1);
          conteoErrores++;
        }

        $$ = getSymbolType($1);

      }
      | CTE_INT {
        $$ = 1;
      }
      | CTE_REAL {
        $$ = 2;
      }
      | A_PARENTESIS expression C_PARENTESIS
      ;
```

- Define factores, que pueden ser identificadores, constantes enteras, constantes reales o expresiones encerradas entre paréntesis.

Writeln_arguments:

```
writeln_arguments: text_expression COMA expression
      | text_expression
      | expression
      ;
```

- Define los argumentos de la instrucción **WRITELN**, que pueden incluir expresiones y textos.

Text_expression:

```
text_expression: CTE_CADENA {
                 $$ = 3;
               }
                | CTE_CHAR {
                 $$ = 3;
               }
                ;
```

- Define expresiones de texto, que pueden ser cadenas de caracteres o caracteres individuales.

Boolean_expression:

```
boolean_expression: boolean_expression AND boolean_member
                    | boolean_expression OR boolean_member
                    | boolean_member
      ;
```

- Define expresiones booleanas que pueden incluir operadores lógicos **AND** y **OR**.

Boolean_member:

```
boolean_member: comparison_expression
      | NOT comparison_expression
      | A_PARENTESIS boolean_expression C_PARENTESIS
      ;
```

- Define booleanos, que pueden ser comparaciones, negaciones o expresiones encerradas entre paréntesis.

Comparison_expression:

```
comparison_expression: expression EQUALS expression
      | expression DISTINTO expression
      | expression MAYOR expression
      | expression MENOR expression
      | expression MAYOR_O_IGUAL expression
      | expression MENOR_O_IGUAL expression
      | A_PARENTESIS comparison_expression C_PARENTESIS
      ;
```

- Define expresiones que comparan dos expresiones aritméticas.

End_section:

```
end_section: END_KEYWORD PUNTO
      {
        printf("Fin del programa.\n");
      }
      ;
```

- Marca el fin del programa con las palabras clave **END** y un punto **“.”.**

Sistema de detección de errores

-El sistema de detección de errores se compone de la función **yyerror**, **addSymbol**, **initializeSymbol** e **isInitialized**.

yyerror maneja los mensajes de error sintácticos, indicando también el número de línea en el que se encuentra el error.

```
int yyerror(const char* msg) {
    printf("Línea %d, ERROR SINTÁCTICO: %s\n", conteoLineas, msg);
    return 0;
}
```

addSymbol se encarga de añadir entradas a la tabla de símbolos y de detectar cuando un identificador ya ha sido declarado anteriormente.

```
void addSymbol(char name[50], int tipo, int cte) {
    if (findSymbol(name) == -1) {
        strcpy(symbolTable[symbolCount].name, name);
        symbolTable[symbolCount].tipo = tipo;
        symbolTable[symbolCount].cte = cte;
        symbolCount++;
    } else {
        printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' ya declarado.\n", conteoLineas, name);
    }
}
```

initializeSymbol se encarga de inicializar el identificador y detecta si no ha sido previamente declarado.

```
void initializeSymbol(char name[50]){
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0) {
            symbolTable[i].inicializado = 1;
        }
    }
}
```

isInitialized comprueba si un símbolo ha sido inicializado y altera un valor int en la estructura Symbol, que luego se usa por la función **initializeSymbol** para comprobar si está inicializado o no.

```
int isInitialized(char name[50]){
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0) {
            if(symbolTable[i].inicializado == 1){
              return 1;
            } else{
              return 0;
            }
        }
    }
}
```

findSymbol busca un símbolo **identificador** en la tabla de símbolos **symbolTable**.

```
struct Symbol symbolTable[100];
int symbolCount = 0;

int findSymbol(char name[50]) {
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0) {
            return i;
        }
    }
    return -1;
}
```

isConstant determina si un identificador dado es una constante.

```
int isConstant(char name[50]){
   for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0) {
            if(symbolTable[i].cte == 1){
              return 1;
            } else{
              return 0;
            }
        }
    }
}
```

getSymbolType obtiene el tipo de un identificador dado desde la tabla de símbolos.

```
int getSymbolType(char name[50]){
   for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0) {
            return symbolTable[i].tipo;
        }
    }
}
```

Adicionalmente se encuentra la sección **%error-verbose**, que nos aporta información adicional sobre el error, por ejemplo, un error pasaría de solo indicarnos en qué línea se encuentra a darnos información sobre qué se esperaba el compilador y qué se encontró en su lugar.

```
%error-verbose

%token PROGRAM BEGIN_KEYWORD END_KEYWORD IF THEN ELSE WHILE FOR TO DOWNTO VAR CONST WRITELN READLN TRUE FALSE
%token EQUALS SUMA RESTA MULT DIVIS DIV MOD MAYOR MENOR MENOR_O_IGUAL MAYOR_O_IGUAL IGUAL DISTINTO AND OR NOT
%token COMA SEMICOLON PUNTO DOS_PUNTOS A_PARENTESIS C_PARENTESIS
%token <tipo> INTEGER REAL STRING BOOLEAN CTE_CADENA CTE_CHAR CTE_INT CTE_REAL
%token <nombre> ID
%%
```

<u>Los tipos de datos usados en la tabla de símbolos son los siguientes:</u>

-Tipo de dato char para identificar el nombre.
-Tipo de dato int para identificar el tipo de símbolo.
-Tipo de dato int para identificar si el identificador es una constante.
-Tipo de dato int para identificar si el identificador ya ha sido inicializado.

<u>Acciones y atributos semánticos usados:</u>

-Tabla de Símbolos:

+Los atributos semánticos serían struct Symbol y symbolTable y su acción es la de guardar la información de las constantes y variables declaradas.

```
struct Symbol {
    char name[50];
    int tipo;
    int cte;
    int inicializado;
};

struct Symbol symbolTable[100];
```

-Contador:

+El atributo es symbolCount y su acción es contar los símbolos utilizados en la tabla.

```
int symbolCount = 0;
```

-Declaración de constantes

+La declaración de constantes se comprende en las siguientes reglas:

-const_item:

+ID IGUAL CTE_CADENA SEMICOLON
+ID IGUAL CTE_CHAR SEMICOLON
+ID IGUAL CTE_INT SEMICOLON
+ID IGUAL CTE_REAL SEMICOLON

```
const_item: ID IGUAL CTE_CADENA SEMICOLON
      {
        printf("Declaración de CONSTANTE DE CADENA.\n");
        addSymbol($1, $3, 1);
      }
      | ID IGUAL CTE_CHAR SEMICOLON
      {
        printf("Declaración de CONSTANTE DE CHAR.\n");
        addSymbol($1, $3, 1);
      }
      | ID IGUAL CTE_INT SEMICOLON
      {
        printf("Declaración de CONSTANTE ENTERA.\n");
        addSymbol($1, $3, 1);
      }
      | ID IGUAL CTE_REAL SEMICOLON
      {
        printf("Declaración de CONSTANTE REAL.\n");
        addSymbol($1, $3, 1);
```

+Su acción es agregar constantes a la tabla.

-Declaración de variables:

+La declaración de variables se comprende en las siguientes reglas:

-var_item:

+ID DOS_PUNTOS INTEGER SEMICOLON
+ID DOS_PUNTOS REAL SEMICOLON
+ID DOS_PUNTOS STRING SEMICOLON
+ID DOS_PUNTOS BOOLEAN SEMICOLON

```
var_item: ID DOS_PUNTOS INTEGER SEMICOLON
      {
            printf("\n Declaración de INTEGER");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS REAL SEMICOLON
      {
            printf("\n Declaración de REAL");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS STRING SEMICOLON
      {
            printf("\n Declaración de STRING");
            addSymbol($1, $3, 0);
      }
      | ID DOS_PUNTOS BOOLEAN SEMICOLON
      {
            printf("\n Declaración de BOOLEAN");
            addSymbol($1, $3, 0);
      }
```

-Inicialización de variables:

+La inicialización de variables se realiza a través de:

-assignment: ID EQUALS expression

+Su acción es comprobar si la variable ha sido declarada previamente mediante la función findSymbol, si no lo ha sido devuelve un error semántico, luego comprueba si la variable ha sido inicializada y si no lo ha sido la inicializa.

+Tambien se comprueba si el tipo de identificador coincide con el tipo de valor que se le quiere asignar, en caso de no coincidir devuelve un error semántico.

+Por último comprueba si es una constante, en caso de serlo manda un error semántico.

```
assignment: ID EQUALS expression
      {
        if (findSymbol($1) == -1){
            printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
            conteoErrores++;
        } else {
          if (getSymbolType($1) != $3){
             printf("Línea %d, ERROR SEMÁNTICO: Tipos incompatibles en la asignación de '%s'.\n", conteoLineas+1, $1);
             conteoErrores++;
          }
        }


       if (isInitialized($1) == 0)
          initializeSymbol($1);

       if (isConstant($1) == 1){
          printf("Línea %d, ERROR SEMÁNTICO: Las constantes no se pueden redefinir\n", conteoLineas+1);
          conteoErrores++;
       }
      }
```

+También se realiza a través de:

-ID EQUALS text_expression

+De nuevo comprueba si el identificador ha sido declarado y comprueba si su tipo coincide con el dato que se le quiere asignar.

```
| ID EQUALS text_expression {
  if (findSymbol($1) == -1){
      printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
      conteoErrores++;
  } else {
    if (getSymbolType($1) != $3){
       printf("Línea %d, ERROR SEMÁNTICO: Tipos incompatibles en la asignación de '%s'.\n", conteoLineas+1, $1);
       conteoErrores++;
    }
  }

 if (isInitialized($1) == 0)
    initializeSymbol($1);

 if (isConstant($1) == 1){
    printf("Línea %d, ERROR SEMÁNTICO: Las constantes no se pueden redefinir\n", conteoLineas+1);
    conteoErrores++;
  }
}
```

-Lectura de variables:

+La lectura de variables se realiza en:

-read_instruction: READLN A_PARENTESIS ID C_PARENTESIS

-Su acción es comprobar si ya ha sido inicializada antes de su uso, si no, la inicializa.

```
read_instruction: READLN A_PARENTESIS ID C_PARENTESIS
        {
          if (isInitialized($3) == 0)
            initializeSymbol($3);
        }
      ;
```

-Uso de variables:

+Su propósito es verificar si las variables utilizadas ya se han declarado e inicializado antes de usarlas.

-Las comprobaciones semánticas se hacen a través de **findSymbol** e **isInitialized**.

```
factor: ID
      {
        if (findSymbol($1) == -1){
          printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no declarado.\n", conteoLineas+1, $1);
          conteoErrores++;
        } else if (isInitialized($1) == 0){
          printf("Línea %d, ERROR SEMÁNTICO: Identificador '%s' no inicializado.\n", conteoLineas+1, $1);
          conteoErrores++;
        }

        $$ = getSymbolType($1);

      }
      | CTE_INT {
        $$ = 1;
      }
      | CTE_REAL {
        $$ = 2;
      }
      | A_PARENTESIS expression C_PARENTESIS
      ;
```

Objetivos no cumplidos

No hemos encontrado la manera por la que el programa se pueda recuperar de un error sintáctico. Si se detecta uno, el programa finaliza y no sigue analizando. Lo más cercano que hemos conseguido ha sido definiendo el token **error** usando después yyerrok e yyclarin, pero los resultados no han sido satisfactorios por lo que hemos excluido la definición token del resto de la gramática.